

Appunti di Programmazione di Dispositivi Mobili

G.C.

Università degli Studi dell'Insubria

Anno Accademico 2025-2026

Indice

1	Introduzione allo Sviluppo Mobile e Android	4
1.1	Il mercato Mobile e lo Sviluppo Cross-Platform	4
1.2	Architettura di un'App Android e Componenti Core	5
1.3	Android Studio e il processo di Build (Gradle)	5
1.3.1	Gradle e Architettura Hardware	5
1.3.2	Dall'Idea all'Esecuzione: APK, DEX e ART	6
1.4	Figma: UI vs UX e Prototipazione	6
1.4.1	Il contenuto di un APK	7
1.5	AndroidManifest.xml	7
2	Il linguaggio Kotlin	9
2.1	Variabili e Tipi Base	9
2.2	Null Safety	9
2.3	Classi e Costruttori (Ordine di esecuzione)	9
2.4	Interfacce vs Classi Astratte	10
2.5	Functional Programming	10
2.5.1	Higher-Order Functions	11
3	Gestione delle Risorse e UI	12
3.1	La cartella res/ e il file R.java	12
3.1.1	Meccanismo di Risoluzione (R.java)	12
3.2	Risoluzione, Densità e le Unità DP e SP	13
3.2.1	La Formula dei DP	13
3.3	Qualificatori e Risorse Alternative	14
3.4	Stili (Styles) vs Temi (Themes)	14
4	Activity, Intent e Ciclo di Vita	16
4.1	Gli Intent	16
4.2	Activity Result API (Risultati dalle Activity)	17
4.3	Il Ciclo di Vita dell'Activity (Lifecycle)	18
4.4	Gestione delle Configuratrion Changes	19

4.5	Tasks e Back Stack	19
5	UI e Liste: Da ListView a Jetpack Compose	21
5.1	Approccio Classico: View e ViewGroup	21
5.1.1	Layout in XML	21
5.2	Le Liste nel Sistema Classico: ListView e Adapters	22
5.2.1	Il concetto di View Recycling (Domanda da Esame!)	22
5.2.2	SimpleAdapter vs ArrayAdapter (Domanda Quiz)	23
5.2.3	L'evoluzione: RecyclerView	23
5.3	Jetpack Compose: Il paradigma moderno	24
5.4	Caricamento di Immagini da Rete	25
6	Fragments e Navigation Component	26
6.1	Introduzione ai Fragment	26
6.2	Il Ciclo di Vita del Fragment	27
6.2.1	Metodi Fondamentali	27
6.3	Gestione della Navigazione: Passato vs Presente	28
6.3.1	Il Vecchio Approccio: FragmentManager e FragmentTransaction	28
6.3.2	Il Nuovo Approccio: Navigation Component	28
7	Jetpack Compose: UI Moderna e Dichiarativa	30
7.1	Da XML a Jetpack Compose	30
7.1.1	Regole base di Compose	30
7.2	Componenti Core e Layout	30
7.2.1	I Modifier	31
7.3	Lo Stato e la Ricomposizione (Cruciale per l'esame)	31
7.4	Le Liste Dinamiche: LazyColumn	32
7.5	Esecuzione Asincrona e Immagini (Side Effects)	33
7.5.1	LaunchedEffect	33
7.5.2	Caricamento Immagini da URL	33
7.6	Navigazione in Compose (Navigation Compose)	33
8	Esecuzione in Background: Services e Alternative	35
8.1	Cos'è un Service?	35
8.2	Le tre tipologie di Service	35
8.3	Ciclo di vita: Started vs Bound	36
8.3.1	Implementare un Started Service (es. Foreground Music Player)	36
8.3.2	Implementare un Bound Service	37
8.4	Modern Android: WorkManager vs Services	37

8.5	Altri Componenti di Background	38
9	Persistenza dei Dati: Room e Firebase	39
9.1	SQLite "Puro" (Il Motore)	39
9.2	Jetpack Room: L'Approccio Moderno	40
9.2.1	Configurazione e KSP	41
9.2.2	Esempio di Codice Room	41
9.3	Backend as a Service: Firebase	42
9.3.1	Realtime Database vs Cloud Firestore	42
9.3.2	Denormalizzazione (Best Practice per DB NoSQL)	42
9.3.3	Lettura Dati e Listener	43

Capitolo 1

Introduzione allo Sviluppo Mobile e Android

1.1 Il mercato Mobile e lo Sviluppo Cross-Platform

Oggi il mercato dei sistemi operativi mobili è dominato da due attori principali: **Android** (Google) che detiene circa il 70-74% del mercato globale, e **iOS** (Apple) con il restante 25-30%.

Quando si decide di sviluppare un'applicazione mobile, ci si trova davanti a un bivio architetturale:

- **Sviluppo Nativo:** Scrivere codice specifico per ogni piattaforma (Kotlin/Java per Android, Swift/Objective-C per iOS). Garantisce le massime performance e accesso immediato alle API del dispositivo.
- **Sviluppo Cross-Platform:** Scrivere il codice una sola volta e deployarlo su più piattaforme. Abbassa i costi e i tempi di sviluppo, ma può portare a compromessi sulle prestazioni e ritardi nel supporto alle nuove API.

I framework Cross-Platform più popolari ad oggi includono:

- **React Native (Facebook):** Basato su JavaScript.
- **Flutter (Google):** Basato sul linguaggio Dart, rendering custom (non usa le UI native ma le ridisegna).
- **Kotlin Multiplatform Mobile (KMM):** Creato da JetBrains. Non è un framework UI, ma un *SDK* che permette di condividere la logica di business (networking, database, logica) in Kotlin, mantenendo lo sviluppo della UI strettamente nativo per Android e iOS.

1.2 Architettura di un'App Android e Componenti Core

Un'applicazione Android non ha un singolo “entry point” (come il classico `main()` in C o Java desktop), ma è composta da quattro blocchi fondamentali (Componenti):

1. **Activities:** Rappresentano le singole schermate con un'interfaccia utente (UI).
2. **Services:** Eseguono operazioni di lunga durata in background, senza UI (es. riproduzione musicale, download).
3. **Broadcast Receivers:** Ascoltano e reagiscono a messaggi o eventi di sistema (es. “batteria scarica”, “schermo sbloccato”).
4. **Content Providers:** Gestiscono e condividono un set di dati tra diverse applicazioni in modo sicuro (es. la rubrica dei contatti).

Lo sviluppo moderno (Modern Android Development) incoraggia l'uso di librerie **Jetpack** (es. ViewModel per lo stato, WorkManager per il background, Room per i DB) che semplificano enormemente la gestione del ciclo di vita di questi componenti.

1.3 Android Studio e il processo di Build (Gradle)

Android Studio è l'IDE ufficiale di Google basato su IntelliJ IDEA. Dal 2015 ha sostituito Eclipse.

1.3.1 Gradle e Architettura Hardware

Nei quiz è importante ricordare che:

- **Gradle** è un sistema di build automation open-source. Costruito sui concetti di Apache Ant e Maven, utilizza un **DSL (Domain-Specific Language)** basato su Groovy o Kotlin.
- I dispositivi mobili utilizzano spesso l'**architettura ARM big.LITTLE**: un'architettura che combina nuclei di processori di dimensioni diverse per **ottimizzare le prestazioni e l'efficienza energetica**.

1.3.2 Dall'Idea all'Esecuzione: APK, DEX e ART

Nei quiz è fondamentale conoscere come il codice diventa un'app eseguibile:

1. Il codice sorgente (Kotlin/Java) viene compilato nel bytecode standard di Java (`.class`).
2. Il compilatore Android converte i file `.class` nel formato **DEX (Dalvik Executable)**, ottimizzato per dispositivi mobili.
3. Il tool **aapt** unisce i file DEX e le risorse compilate per creare l'archivio **APK** (Android Application Package).

Android Runtime (ART) vs Dalvik

In passato, Android usava la macchina virtuale **Dalvik** (compilazione JIT - Just In Time). Oggi utilizza **ART (Android Runtime)**. ART compila l'app in codice macchina nativo al momento dell'installazione tramite la tecnica **AOT (Ahead-Of-Time) compilation**.

Vantaggi: Esecuzione molto più veloce.

Svantaggi: Tempi di installazione leggermente più lunghi e maggiore spazio occupato sul disco.

1.4 Figma: UI vs UX e Prototipazione

Prima di scrivere il codice, la fase di design è fondamentale.

- **UX (User Experience):** Struttura logica, gerarchie, posizionamento e flussi funzionali. Analizza i comportamenti reali degli utenti.
- **UI (User Interface):** Struttura visiva, colori, font, icone. Assicura che il prodotto sia visivamente stimolante.

Figma nei Quiz

Figma è una piattaforma di design collaborativo **basata su cloud** (Web-based tramite WebGL/WASM). Utilizza i **CRDT** (Conflict-free Replicated Data Types) e **WebSockets** per permettere a più persone di lavorare sullo stesso documento in *real-time* senza conflitti di file (approccio *Last-Writer-Wins*). Consente di creare **veri prototipi** per testare l'esperienza utente prima di scrivere codice.

- **Caratteristica principale:** È *Web-based* (WASM + WebGL). A differenza di software storici come Sketch, non richiede installazioni locali o file `.sketch` sul disco.
- Il motore di rendering sotto il cofano è scritto in **C++** e transpilato in WebAssembly, permettendo la gestione di grafica vettoriale con la potenza di un videogioco direttamente nel browser.

1.4.1 Il contenuto di un APK

Da un punto di vista dei file, un APK è un archivio ZIP che contiene:

- `classes.dex`: Il codice eseguibile dell'applicazione.
- `res/`: Le risorse non compilate.
- `resources.arsc`: Le risorse compilate (tabelle di lookup generate da `aapt`).
- `META-INF/`: Informazioni sulle firme digitali.
- `AndroidManifest.xml`: Il file fondamentale che descrive l'app al sistema operativo Android.

1.5 AndroidManifest.xml

Ogni app Android deve avere un file chiamato `AndroidManifest.xml` nella root del progetto. Definisce le caratteristiche essenziali dell'applicazione necessarie ad Android prima ancora che l'app venga avviata:

- Dichiarare il *package name* (che funge da ID univoco dell'app).
- **Dichiara i Componenti:** Tutte le `<activity>`, `<service>`, `<receiver>` e `<provider>` devono essere dichiarati qui. Se crei una classe Activity in Kotlin ma non la dichiari nel Manifest, l'app andrà in crash se cerchi di avviarla.

- Dichiarare i **permessi** (<uses-permission>) richiesti (es. INTERNET, fotocamera).
- Dichiarare le caratteristiche hardware richieste (es. touch screen, NFC).

Un costrutto chiave all'interno del manifest è l'**Intent-Filter**. Per indicare al sistema operativo qual è la prima Activity da lanciare all'avvio dell'app (l'Activity *Home*), si usa la seguente sintassi:

```
1 <activity android:name=".MainActivity" android:exported="true">
2   <intent-filter>
3     <action android:name="android.intent.action.MAIN" />
4     <category android:name="android.intent.category.LAUNCHER"
5       />
6   </intent-filter>
</activity>
```

Il flag `android:exported="true"` indica che questa Activity può essere avviata da app esterne (incluso il launcher (la schermata home) del sistema operativo).

Capitolo 2

Il linguaggio Kotlin

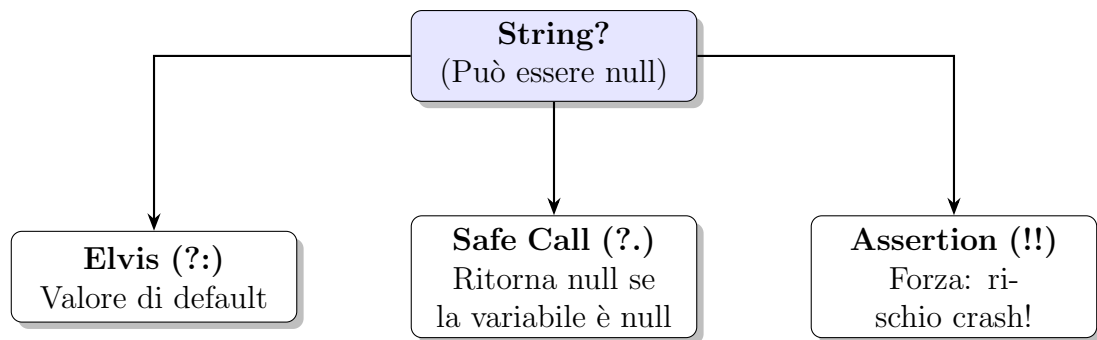
2.1 Variabili e Tipi Base

In Kotlin **tutto è un oggetto** (non esistono tipi primitivi come in Java). Il compilatore non effettua conversioni implicite (es. non puoi assegnare un `Int` a un `Long` senza `.toLong()`).

- `val`: Immutabile (sola lettura).
- `var`: Mutabile.

2.2 Null Safety

Per evitare il crash dell'app (`NullPointerException`), Kotlin introduce tipi nullable.



2.3 Classi e Costruttori (Ordine di esecuzione)

Una tipica domanda da quiz riguarda l'ordine esatto di inizializzazione quando si istanzia una classe in Kotlin. L'ordine rigoroso è:

1. **Costruttore Primario:** Definito nella firma della classe. Se non viene specificato, Kotlin ne crea uno implicito vuoto.
2. **Proprietà e blocchi init:** Vengono eseguiti dall'alto verso il basso, **esattamente nell'ordine in cui compaiono** nel codice. Il blocco `init` non è un costruttore, ma fa parte dell'inizializzazione legata al costruttore primario, ottimo per fare validazioni (es. `require(prezzo >= 0)`).
3. **Costruttore Secondario:** Definito dalla keyword `constructor`. Eseguito per ultimo.

Regola d'oro Costruttori (Domanda Quiz)

Il costruttore secondario **deve sempre delegare** al primario usando `this(...)` (se il primario esiste). Se si omette questa delega, si ottiene un **errore di compilazione**.

2.4 Interfacce vs Classi Astratte

Caratteristica	Interfaccia	Classe Astratta
Stato	No (no variabili istanza)	Sì (variabili istanza)
Ereditarietà	Molteplice	Singola
Metodi	Astratti o con default	Astratti o concreti

2.5 Functional Programming

Kotlin permette di trattare le funzioni come variabili.

- **Lambda:** Funzioni anonime `{ a, b -> a + b }`.
- **Uso di it:** Se la lambda ha un solo parametro, puoi omettere il nome e usare la keyword `it`.

Listing 2.1: Esempio di Lambda e Filter

```

1 val numeri = listOf(3, 8, 12, 5)
2 // Versione estesa
3 val maggiori7 = numeri.filter { n -> n > 7 }
4 // Versione compatta con 'it'
5 val maggiori7Compatta = numeri.filter { it > 7 }

```

Funzioni sulle Collezioni (Viste nei Quiz)

Nei quiz pratici del corso appaiono spesso queste funzioni della standard library:

- **zip**: Unisce due collezioni in una lista di coppie (Pairs). Es: `str1.zip(str2)` utile per calcolare la distanza di Hamming ciclando sulle coppie di lettere.
- **count { }**: Restituisce il numero di elementi che soddisfano una condizione.
- **map { }**: Applica una trasformazione a ciascun elemento restituendo una nuova lista.

2.5.1 Higher-Order Functions

Una **Higher-Order Function** è una funzione che riceve un'altra funzione come parametro oppure restituisce una funzione. È una caratteristica fondamentale di Kotlin. Nel parametro, la firma della funzione viene definita tramite i tipi di ingresso e di uscita. Ad esempio, `operazione: (Int, Int) -> Int` accetta una funzione (o lambda) che prende due interi e ne restituisce uno.

Listing 2.2: Esempio Quiz: Calcolatrice

```
1 fun calcola(a: Int, b: Int, operazione: (Int, Int) -> Int): Int {  
2     return operazione(a, b)  
3 }  
4  
5 fun main() {  
6     // Passo una lambda come parametro  
7     val risultato = calcola(6, 3) { x, y -> x * y }  
8     println(risultato) // Stampa 18  
9 }
```

Capitolo 3

Gestione delle Risorse e UI

Un'applicazione Android è formata da Codice (Java/Kotlin) e **Risorse**. Le risorse sono tutto ciò che non è codice logico: file XML di layout, pacchetti lingua, immagini, file audio, stili e colori. L'obiettivo è separare la presentazione (Layout) dalla logica (Management).

3.1 La cartella `res/` e il file `R.java`

Le risorse vengono salvate nella cartella `res/` e suddivise in sottocartelle specifiche:

- `layout/`: file XML che definiscono le interfacce grafiche.
- `values/`: contiene file XML che memorizzano valori semplici come stringhe (`strings.xml`), colori (`colors.xml`), dimensioni (`dimens.xml`) e stili (`styles.xml`).
- `drawable/`: per immagini (.png, .jpg), grafiche vettoriali o shape XML.
- `mipmap/`: dedicato esclusivamente alle icone dell'applicazione (launcher icons).

3.1.1 Meccanismo di Risoluzione (`R.java`)

Al momento della build, il tool `aapt` assegna ad ogni risorsa un ID intero univoco in formato esadecimale (es. `0x7f05000b`) e lo memorizza in una classe autogenerata chiamata `R.java`.

- Da XML ci si riferisce alle risorse con la sintassi: `@tipo/nome` (es. `@string/app_name`).
- Da codice Kotlin/Java ci si riferisce con: `R.tipo.nome` (es. `R.id.button1`).

3.2 Risoluzione, Densità e le Unità DP e SP

Android gira su migliaia di dispositivi diversi. L'applicazione deve mantenere l'indipendenza dalla densità dello schermo ("Density Independence").

Definizioni Chiave per i Quiz

Resolution (Risoluzione): Numero totale di pixel dello schermo (es. 1920x1080). Due schermi delle stesse dimensioni fisiche possono avere risoluzioni diverse.

Density (DPI / PPI): Punti per pollice. Misura quanto i pixel sono vicini fisicamente l'uno all'altro.

Se creassimo un pulsante largo "100 pixel" (100px), questo apparirebbe grande su un vecchio dispositivo e minuscolo su uno smartphone moderno ad altissima definizione. Per questo si usano i **Density Independent Pixels (DP)**.

3.2.1 La Formula dei DP

La baseline di Android è fissata a una densità media (**mdpi** = 160 dpi). In questa specifica densità, **1 dp = 1 px**. La formula per convertire da DP a Pixel hardware è:

$$px = dp \times \left(\frac{dpi}{160} \right) \quad (3.1)$$

Esempio: su uno schermo **xxhdpi** (480 dpi), un elemento di 10 dp corrisponderà a $10 \times (480/160) = 30px$ fisici.

Per i testi si utilizza invece **SP (Scale-independent Pixels)**, che si comporta matematicamente come il DP, ma scala dinamicamente in base alle preferenze di dimensione del font impostate dall'utente nel sistema operativo.

Calcolo della dimensione fisica (Quiz)

Nei quiz viene chiesto di calcolare la dimensione fisica in pollici (*inches*) di un elemento. La formula è:

$$\text{Physical Size (in)} = \frac{\text{Pixel Reali}}{\text{DPI del dispositivo}}$$

Ricordando che $\text{Pixel Reali} = \text{DP} \times \left(\frac{\text{DPI}}{160} \right)$.

Esempio: 100 dp su uno schermo a 240 dpi.

1) $\text{Pixel} = 100 \times (240/160) = 150px$.

2) $\text{Physical Size} = 150/240 = 0.625$ pollici.

3.3 Qualificatori e Risorse Alternative

Per gestire dispositivi diversi (es. orientamento, lingua), si creano cartelle "alternative" specificando un **qualificatore** nel nome della cartella: `<nome_risorsa>-<qualificatore>`. Esempi:

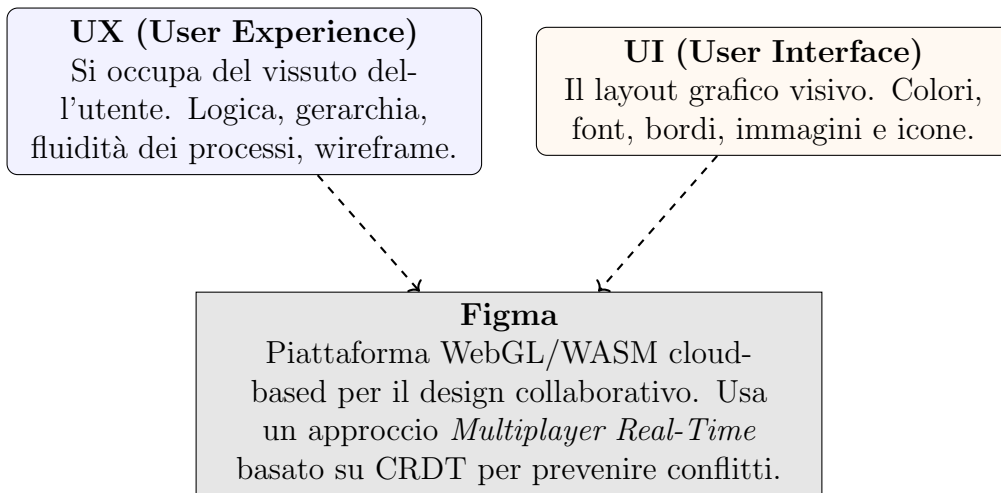
- `res/layout/` (Layout di default per smartphone portrait)
- `res/layout-land/` (Layout che Android carica automaticamente se si ruota il telefono)
- `res/values-en/` (Stringhe in lingua inglese)

A runtime, Android cerca il "best match". Se un qualificatore richiesto contraddice la configurazione del dispositivo (es. cerco layout per `fr` ma l'app gira in `en-GB`), Android elimina quelle directory e usa i fallback di default.

3.4 Stili (Styles) vs Temi (Themes)

Entrambi si definiscono tramite i tag `<style>` dentro `res/values/styles.xml`. Possono ereditare proprietà da un genitore tramite l'attributo `parent`. La differenza sta nell'**applicazione**:

- **Style:** Viene applicato a una singola `View` (es. un bottone) tramite l'attributo `style="@style/MioStile"`.
- **Theme:** È un'estensione dello style, ma viene applicato all'intera applicazione o a una singola `Activity` nel file `AndroidManifest.xml` (es. `android:theme="@style/MioTe`). Un tema modifica parametri globali come la barra di navigazione, lo status bar o i colori primari.



Capitolo 4

Activity, Intent e Ciclo di Vita

L'**Activity** è uno dei mattoni fondamentali (*building blocks*) delle app Android. Rappresenta una singola schermata con un'interfaccia utente. Funge da punto di ingresso (*entry point*) per l'interazione dell'utente con l'app.

4.1 Gli Intent

Un **Intent** è un oggetto di messaggistica (*message-passing framework*) che permette a componenti diversi di richiedere azioni ad altri componenti, sia all'interno della stessa app che verso app esterne.

Possono essere di due tipi principali:

- **Intent Espliciti (Explicit Intents):** Specificano *esattamente* la classe del componente da avviare (es. `ActivityB::class.java`). Si usano quasi sempre all'interno della propria app.
- **Intent Impliciti (Implicit Intents):** Non specificano il componente esatto, ma indicano un'**azione** generica (es. `ACTION_VIEW`) e dei dati (es. un URL). Il sistema Android verifica gli *Intent-Filter* delle app installate e sceglie quella adatta (o chiede all'utente quale usare).

Listing 4.1: Esempi di Intent

```
1 // 1. Intent Esplicito
2 val intentEsplicito = Intent(this, SecondActivity::class.java)
3 startActivity(intentEsplicito)
4
5 // 2. Intent Implicito (aprire una pagina web)
6 val uri = Uri.parse("http://www.uninsubria.it")
7 val intentImplicito = Intent(Intent.ACTION_VIEW, uri)
8 startActivity(intentImplicito)
```

4.2 Activity Result API (Risultati dalle Activity)

Spesso un'Activity ne avvia un'altra per ottenere un risultato (es. scattare una foto, scegliere un contatto).

Attenzione all'esame: Deprecazione!

Il vecchio metodo `startActivityResult()` e il relativo callback `onActivityResult()` sono **deprecati**. Erano problematici perché, se l'app veniva messa in background e uccisa dal sistema, il ritorno del risultato si perdeva.

Il nuovo standard è l'**Activity Result API** (da `androidx 1.2.0`), che si divide in 3 passaggi fondamentali:

1. **Contract:** Definisce input e output (es. `StartActivityResult()`).
2. **Register:** Si registra il contratto nella fase di inizializzazione chiamando `registerForActivityResult()`, che restituisce un `ActivityResultLauncher`.
3. **Launch:** Si invoca il metodo `launch()` sull'oggetto `Launcher` quando si vuole avviare l'azione.

Listing 4.2: Ottenere un risultato con Activity Result API (Esatto da Quiz)

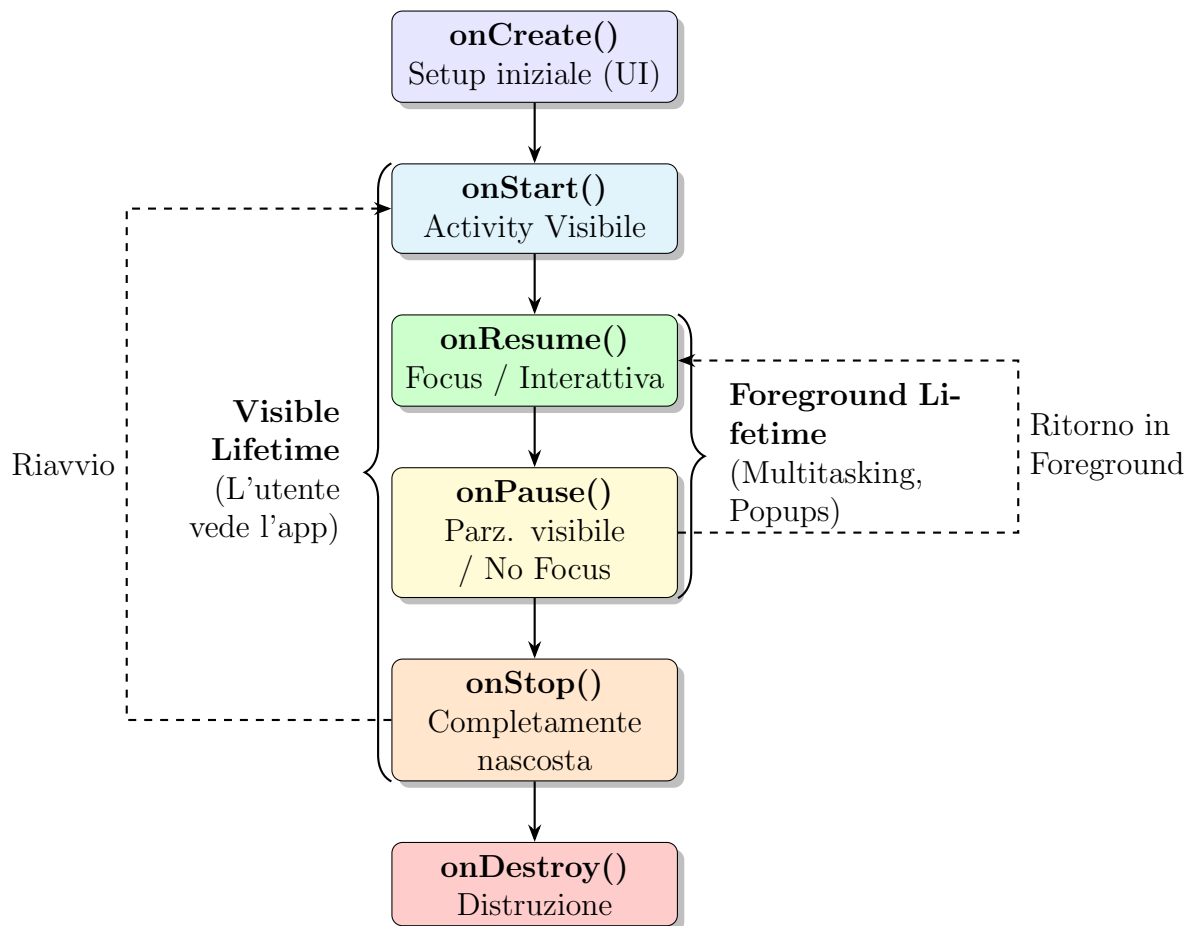
```
1 // 1. Registrazione del contratto (da fare come attributo della
   classe)
2 val launcher = registerForActivityResult(
3     ActivityResultContracts.StartActivityResult()
4 ) { result ->
5     // Questo e' il blocco dell'ActivityResultCallback
6     if (result.resultCode == Activity.RESULT_OK) {
7         val data = result.data?.getStringExtra("RISPOSTA")
8     }
9 }
10
11 // 2. Lancio dell'Intent (es. dentro un listener)
12 button.setOnClickListener {
13     val intent = Intent(this, SecondActivity::class.java)
14     launcher.launch(intent) // L'istruzione per far partire la
   richiesta
15 }
```

Perché `startActivityForResult` è deprecato?

Il vecchio metodo è stato deprecato perché il codice era strettamente accoppiato al ciclo di vita dell'Activity, generando bug (es. crash o perdita di dati) se l'Activity chiamante veniva distrutta dal sistema mentre la seconda Activity era aperta.

4.3 Il Ciclo di Vita dell'Activity (Lifecycle)

Comprendere il ciclo di vita è essenziale. Android notifica i cambiamenti di stato dell'Activity invocando metodi di *callback*.



Dettagli chiave sui metodi:

- `onCreate()`: Chiamato una sola volta per istanziare l'UI (`setContentView`).

- `onResume()` / `onPause()`: Segnano i confini del *Foreground lifetime*. In `onPause` l'app è ancora visibile ma non ha il focus (es. in modalità Multi-finestra, o se c'è un Dialog semi-trasparente sopra).
- `onStop()`: L'app non è più visibile (es. utente preme il tasto Home o apre un'altra app a schermo intero).

4.4 Gestione delle Configuratrion Changes

Quando avviene un cambiamento di configurazione a runtime (es. **rotazione dello schermo**, cambio della lingua, cambio modalità dark/light), il comportamento di default di Android è **distruggere e ricreare l'Activity** (`onPause` → `onStop` → `onDestroy` → `onCreate`).

Il Problema: Questo fa perdere tutte le variabili locali e lo stato non salvato. Le soluzioni in ordine di correttezza (architetturale):

1. **ViewModel (MVVM):** È l'approccio standard moderno. Il `ViewModel` "sopravvive" alle rotazioni. Lo stato della UI viene tenuto nel `ViewModel`.
2. `onSaveInstanceState()` e `onRestoreInstanceState()`: Si salva un `Bundle` chiave-valore e lo si recupera alla ricreazione.
3. **Forzare il Manifest** (`android:configChanges`): Specificando nel Manifest `android:configChanges="orientation|screenSize"`, Android *non* riavvierà l'Activity, ma chiamerà `onConfigurationChanged()`. Questa pratica **non è consigliata** se non strettamente necessaria.

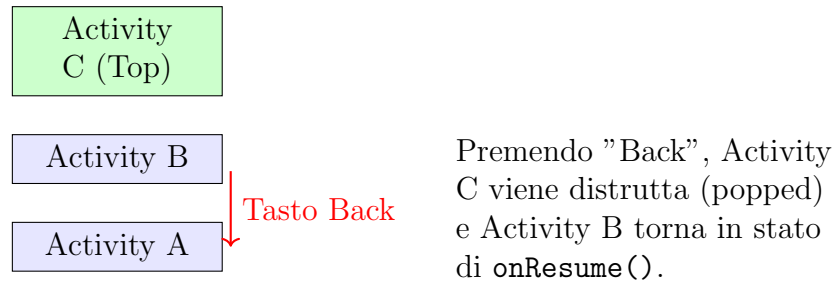
4.5 Tasks e Back Stack

Un **Task** è un insieme di Activity con cui l'utente interagisce per svolgere un lavoro. Queste Activity sono organizzate in uno stack (LIFO) chiamato **Back Stack**.

Quando si avvia un'Activity con un Intent, si possono usare dei **Flags** per alterare il comportamento standard del Back Stack:

- `Intent.FLAG_ACTIVITY_NEW_TASK`: Avvia l'Activity in un nuovo task (o la porta in primo piano se il task esiste già). Indica che l'Activity funge da inizio di un nuovo lavoro.
- `Intent.FLAG_ACTIVITY_CLEAR_TASK`: Causa la rimozione di tutte le Activity precedenti dal task corrente prima dell'avvio della nuova. Di solito usato in

combinazione con `NEW_TASK` (es. dopo un logout per impedire all'utente di tornare indietro).



Comportamento dei Flag (Domande Quiz)

- `FLAG_ACTIVITY_NEW_TASK` e `FLAG_ACTIVITY_CLEAR_TASK` usati insieme: Vengono rimosse tutte le Activity precedenti dal Task corrente e la nuova Activity diventa la root. Premendo "Back" si uscirà dall'app.
- Se l'utente preme il tasto Back: l'Activity in cima al Back Stack (foreground) viene distrutta (`onDestroy`) e la precedente torna in `onResume`.

Capitolo 5

UI e Liste: Da ListView a Jetpack Compose

La costruzione di interfacce grafiche in Android ha subito un'enorme evoluzione. È vitale per l'esame conoscere sia l'approccio classico (XML e View) che quello moderno dichiarativo (Jetpack Compose).

5.1 Approccio Classico: View e ViewGroup

Il sistema classico si basa sull'albero degli oggetti UI, un'implementazione del **Design Pattern Composite**.

- **View (Component):** La classe base per tutti gli elementi grafici interattivi (TextView, Button, ecc.). Implementa la funzione `onDraw()`.
- **ViewGroup (Composite):** Sottoclasse di **View** che funge da contenitore (Layout) per altre View o ViewGroup (es. `LinearLayout`, `FrameLayout`).

5.1.1 Layout in XML

Tra i ViewGroup più usati:

- **LinearLayout:** Dispone gli elementi in una singola riga o colonna. Usa la proprietà `android:layout_weight` per definire in proporzione *quanto spazio extra* deve occupare un figlio rispetto agli altri.
- **ConstraintLayout:** Layout piatto (flat), basato su vincoli (constraint) tra gli elementi. Molto performante per UI complesse rispetto a `LinearLayout` innestati.

- **FrameLayout:** Progettato per contenere un singolo elemento principale. Se ha più figli, li **sovrappone** uno sopra l'altro (funziona da "cornice").

5.2 Le Liste nel Sistema Classico: ListView e Adapters

Per mostrare dati dinamici (es. rubrica contatti) non si possono creare migliaia di `TextView` manualmente. Si usano i componenti `AdapterView` (`ListView`, `GridView`) accompagnati dagli **Adapter**.

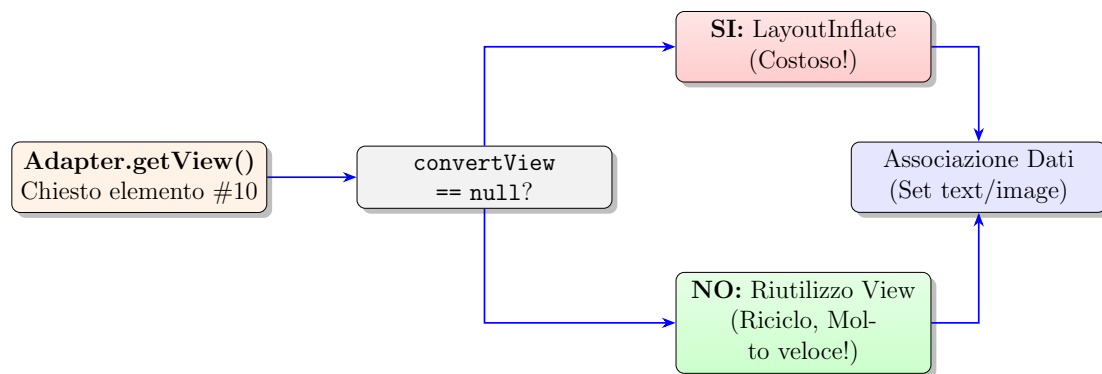
L'**Adapter** è una classe non-visiva che funge da "ponte" tra l'origine dei dati (`Array`, `Database`) e la `ListView`. I principali sono `ArrayAdapter`, `CursorAdapter` e il generico `BaseAdapter`.

5.2.1 Il concetto di View Recycling (Domanda da Esame!)

Creare nuovi oggetti in Java/Kotlin (*Object creation*) è un'operazione "costosa" in termini di memoria e CPU, e sollecita troppo il *Garbage Collector*. Per questo motivo, la `ListView` istanzia in RAM **solo il numero di righe strettamente necessarie** a riempire lo schermo, più un paio di scorta.

Quando l'utente fa scroll verso il basso:

1. La riga che esce dallo schermo in alto finisce in una coda di riciclo (`Scrap View`).
2. L'Adapter, nel suo metodo `getView(position, convertView, parent)`, riceve come parametro `convertView` la view appena uscita dallo schermo.
3. Se `convertView != null`, significa che possiamo **riutilizzare le View esistenti**. Non facciamo un nuovo "inflate" (che consumerebbe CPU e RAM), ma ci limitiamo a cambiare i testi/immagini al suo interno. Questo riduce il consumo di memoria e migliora vertiginosamente le performance.



5.2.2 SimpleAdapter vs ArrayAdapter (Domanda Quiz)

Mentre l'`ArrayAdapter` viene utilizzato principalmente quando i dati sono contenuti in una **lista semplice di oggetti** o array, il `SimpleAdapter` si usa per dati più complessi.

- `SimpleAdapter` è più flessibile perché **può associare più campi a diverse View** (es. un titolo, un sottotitolo e un'immagine nella stessa riga).
- Viene usato quando i dati sono **strutturati come Map (chiave-valore)**, tipicamente in una `List<Map<String, Any>>`.
- L'array di stringhe fornito nel suo costruttore rappresenta **le chiavi della Map**, mentre l'array di interi rappresenta **gli ID delle View nel layout** in cui inserire i dati.
- Se si tenta di caricare un'immagine da un URL web tramite `SimpleAdapter`, l'immagine **non viene caricata automaticamente**. È necessario implementare un `ViewBinder` personalizzato e usare librerie come Glide.

5.2.3 L'evoluzione: RecyclerView

Essendo la `ListView` limitata, Google ha introdotto la `RecyclerView`.

- **Gestione del Layout:** Delega il posizionamento a un `LayoutManager` (es. `LinearLayoutManager` per liste, `GridLayoutManager` per griglie).
- **Performance:** Impone (forza) per architettura l'uso del pattern **ViewHolder**, obbligando lo sviluppatore a mantenere i riferimenti in memoria degli elementi della riga, rendendo il riciclo estremamente efficiente.

5.3 Jetpack Compose: Il paradigma moderno

Oggi, Google spinge **Jetpack Compose**, un toolkit UI **dichiarativo** interamente scritto in Kotlin. Non ci sono più file XML, non c'è più 'findViewById', né si usano gli Adapter!

In Compose:

- **Tutto è una funzione:** Le UI si creano scrivendo funzioni annotate con `@Composable`. I nomi di queste funzioni si scrivono in PascalCase (es. `fun MyButton()`).
- **UI come funzione dello Stato:** `UI = f(state)`. Se lo stato (i dati) cambia, Compose effettua automaticamente la **Ricomposizione** (*Recomposition*), ridisegnando solo le parti di UI colpite dal cambiamento.
- **Gestione di liste lunghe:** Al posto di `RecyclerView`, si usano `LazyColumn` (lista verticale) e `LazyRow` (orizzontale). Queste funzioni creano componenti solo quando diventano visibili sullo schermo, garantendo le stesse performance della `RecyclerView`, ma in 3 righe di codice.

Confronto XML vs Compose (Utile per i quiz)

- `LinearLayout (vertical)` → `Column`
- `LinearLayout (horizontal)` → `Row`
- `FrameLayout` → `Box`
- `RecyclerView` → `LazyColumn` / `LazyRow`
- `ListView.setOnItemClickListener` → `Modifier.clickable { }`

Listing 5.1: Creare una lista in Jetpack Compose

```
1  @Composable
2  fun ListaUtenti(utenti: List<String>) {
3      // LazyColumn equivale a una RecyclerView efficiente
4      LazyColumn {
5          // Il costrutto 'items' sostituisce l'Adapter!
6          items(utenti) { utente ->
7              Text(
8                  text = utente,
9                  modifier = Modifier
10                     .fillMaxWidth()
11                     .padding(16.dp)
12              )
13          }
14      }
15  }
```

```
13     }
14   }
15 }
```

5.4 Caricamento di Immagini da Rete

Sia con XML che con Compose, scaricare immagini da Internet sul *Main Thread* farebbe bloccare o crashare l'app. Nelle app in Compose moderne, al posto di gestire thread separati, si utilizza la libreria **Coil** (Coroutine Image Loader), usando direttamente il widget **AsyncImage**:

Listing 5.2: AsyncImage con Coil

```
1 AsyncImage(
2     model = "https://picsum.photos/200/300",
3     contentDescription = "Immagine di prova",
4     modifier = Modifier.size(64.dp)
5 )
```

AsyncImage esegue il download in modo asincrono senza bloccare l'interfaccia, salva l'immagine in cache e la mostra appena pronta. Ricorda che è sempre necessario il permesso **INTERNET** nell' **AndroidManifest.xml**.

Capitolo 6

Fragments e Navigation Component

6.1 Introduzione ai Fragment

Un **Fragment** rappresenta **una porzione di interfaccia utente all'interno di un'Activity**. Migliorano la riusabilità del codice e permettono di creare interfacce adattive per diversi dispositivi. Nel ciclo di vita, il metodo utilizzato per creare la UI di un Fragment è `onCreateView()`, ed è al suo interno che il layout XML viene caricato tramite il metodo `inflate()`.

Per ospitare visivamente un Fragment in un layout XML, si utilizza il componente `FragmentManager`.

I Fragment offrono tre vantaggi principali:

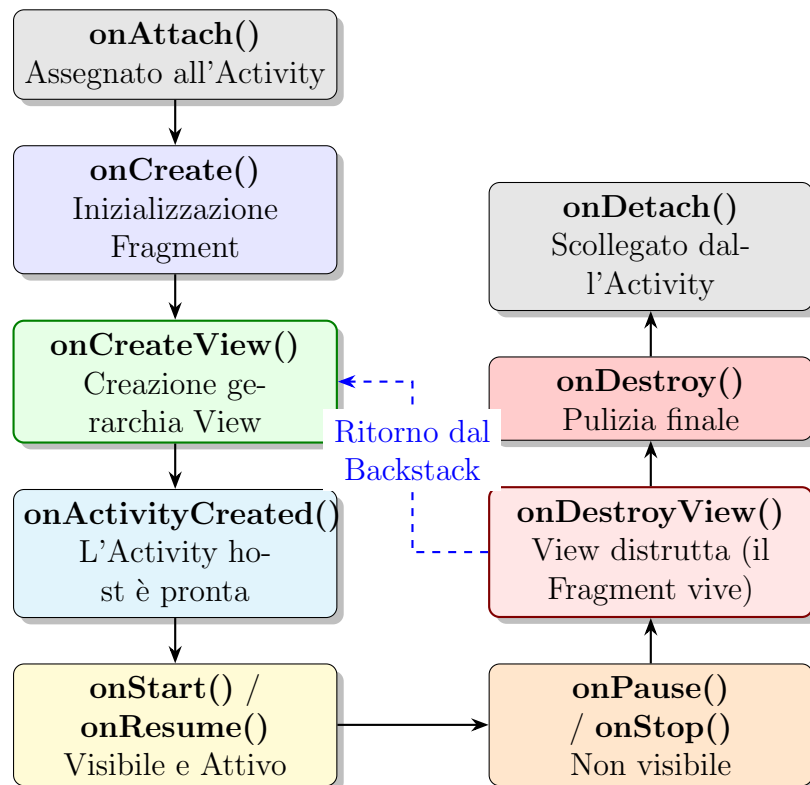
- **Modularità:** Dividono il codice complesso di un'Activity in parti più piccole e gestibili.
- **Riusabilità:** Permettono di riutilizzare la stessa porzione di UI (es. un menù laterale) in più Activity.
- **Adattabilità:** Consentono di mostrare layout diversi a seconda dell'orientamento o delle dimensioni dello schermo (es. Master-Detail flow su tablet).

Nota per l'esame

Un Fragment **non può esistere da solo**. Deve essere **sempre** ospitato (embedded) all'interno di un'Activity. Il suo ciclo di vita è direttamente influenzato da quello dell'Activity host.

6.2 Il Ciclo di Vita del Fragment

Il ciclo di vita di un Fragment è simile a quello di un'Activity, ma con alcuni metodi aggiuntivi cruciali. Una tipica domanda d'esame riguarda la differenza tra la distruzione del Fragment e la distruzione della sua View.



6.2.1 Metodi Fondamentali

- **onCreateView()**: È qui che si effettua l'*inflate* del layout XML del fragment. Il metodo **deve ritornare la View** creata.
- **onDestroyView()**: La View associata al Fragment viene rimossa e distrutta. *Attenzione:* Il Fragment stesso è ancora in memoria (potrebbe essere nel backstack)!
- **onAttach() / onDetach()**: Segnano il momento in cui il Fragment viene collegato o scollegato dall'Activity madre.

Listing 6.1: Creazione classica di un Fragment

```
1 class ExampleFragment : Fragment() {
```

```

2      override fun onCreateView(
3          inflater: LayoutInflater,
4          container: ViewGroup?,
5          savedInstanceState: Bundle?
6      ): View? {
7          // Inflate del layout XML per questo fragment
8          return inflater.inflate(R.layout.fragment_example,
9                                  container, false)
10     }

```

6.3 Gestione della Navigazione: Passato vs Presente

6.3.1 Il Vecchio Approccio: FragmentManager e FragmentTransaction

Prima di Jetpack, cambiare Fragment richiedeva codice verboso e pronò a errori. Si utilizzava il `FragmentManager` per avviare una `FragmentTransaction`:

Listing 6.2: Vecchio approccio (Sconsigliato)

```

1  val transaction = supportFragmentManager.beginTransaction()
2  transaction.replace(R.id.fragmentContainerView, DetailFragment())
3  transaction.addToBackStack(null) // Per gestire il tasto Indietro
4  transaction.commit()

```

6.3.2 Il Nuovo Approccio: Navigation Component

Oggi lo standard è l'uso del **Navigation Component** di Jetpack. Elimina la necessità di scrivere transazioni manuali e gestisce automaticamente il *Back Stack* (la pila delle schermate precedenti).

Si basa su 3 elementi chiave:

1. **NavGraph (XML):** Un file XML (nella cartella `res/navigation`) in cui si definiscono tutte le destinazioni (Fragment) e le azioni (freccie di navigazione) tra di esse.
2. **NavHost (FragmentManager):** Il contenitore vuoto all'interno del layout dell'Activity (`activity_main.xml`) in cui i Fragment vengono scambiati.

3. **NavController (Kotlin):** L'oggetto che nel codice gestisce effettivamente il cambio di schermata.

Listing 6.3: Navigazione con NavController

```
1 // Nel bottone del Fragment di partenza:
2 button.setOnClickListener {
3     // Si naviga usando l'ID dell'azione o l'ID del fragment
4     // destinazione definito nel NavGraph
5     findNavController().navigate(R.id.
        action_homeFragment_to_detailFragment)
}
```

Domanda Tipica d'Esame

Qual è il componente che si occupa di gestire materialmente lo swap dei fragment e il back stack nel Navigation Component?

Risposta: Il **NavController**. Il *NavHost* è solo il contenitore visivo, mentre il *NavGraph* è solo la mappa (regole) della navigazione.

Capitolo 7

Jetpack Compose: UI Moderna e Dichiarativa

7.1 Da XML a Jetpack Compose

Storicamente, Android usava un approccio **Imperativo** per la UI: si creava un albero di View in XML e si modificava il loro stato tramite codice Kotlin/Java (usando `findViewById`, `setText`, ecc.).

Jetpack Compose sposa il paradigma **Dichiarativo**. L'interfaccia utente è scritta interamente in Kotlin tramite funzioni. Lo sviluppatore descrive *cosa* la UI deve mostrare in base allo stato attuale, e Compose si occupa di aggiornarla automaticamente quando lo stato cambia.

7.1.1 Regole base di Compose

- **Annotazione `@Composable`:** Qualsiasi funzione che emette UI deve essere annotata con `@Composable`.
- **PascalCase:** Per convenzione, le funzioni composable che fungono da widget si scrivono con l'iniziale maiuscola (es. `Greeting()`, `HomeScreen()`), comportandosi di fatto come se fossero delle classi grafiche.
- **Nessun XML:** Tutta la UI risiede in file `.kt`.

7.2 Componenti Core e Layout

I layout classici in XML hanno equivalenti diretti in Compose:

Mondo XML (Imperativo)	Mondo Compose (Dichiarativo)
LinearLayout (vertical)	Column { }
LinearLayout (horizontal)	Row { }
FrameLayout (Sovrapposizione)	Box { }
TextView	Text("Stringa")
RecyclerView / ListView	LazyColumn { } / LazyRow { }
setOnClickListener { }	Modifier.clickable { } oppure Button(onClick=...) { }

7.2.1 I Modifier

In Compose non esistono attributi XML come `android:layout_width`. Tutto ciò che riguarda lo stile, le dimensioni, il padding e i comportamenti (click, scroll) viene passato tramite il parametro `Modifier`.

Listing 7.1: Esempio pratico di Modifier

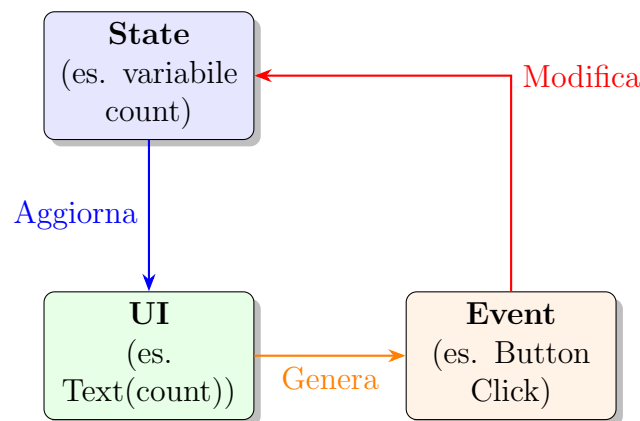
```

1 Text(
2     text = "Hello_PDM!",
3     modifier = Modifier
4         .fillMaxWidth()           // Equivalente a match_parent
5         .padding(16.dp)           // Margine/Padding
6         .clickable { /* fai qualcosa */ }
7 )

```

7.3 Lo Stato e la Ricomposizione (Cruciale per l'esame)

In Compose, la UI è una funzione dello Stato: $UI = f(state)$.



Se una semplice variabile Kotlin (es. `var count = 0`) cambia, la UI **non** si aggiorna. Per far capire a Compose che deve ri-disegnare la schermata, dobbiamo usare **State** e **remember**.

- **mutableStateOf(valore):** Crea un contenitore osservabile. Quando il valore cambia, Compose viene "notificato".
- **remember { }:** Dice a Compose di "ricordare" questo valore tra una ricomposizione e l'altra, altrimenti ad ogni refresh la variabile verrebbe re-inizializzata al valore di partenza.
- **Ricomposizione (Recomposition):** È il processo con cui Compose ri-esegue le funzioni `@Composable` i cui dati di stato sono cambiati, aggiornando solo i pezzi di schermo necessari.

Listing 7.2: Gestione corretta dello Stato

```
1 @Composable
2 fun Counter() {
3     // 1. Dichiariamo lo stato
4     var count by remember { mutableStateOf(0) }
5
6     Button(onClick = {
7         // 2. Modifichiamo lo stato (scatena la Ricomposizione)
8         count++
9     }) {
10        // 3. La UI si aggiorna automaticamente
11        Text("Click: 📄$count")
12    }
13 }
```

7.4 Le Liste Dinamiche: LazyColumn

Mentre nel vecchio sistema serviva creare una `RecyclerView`, un `Adapter`, un `ViewHolder` e un layout XML per la riga, in Compose le liste sono banali.

`LazyColumn` (e `LazyRow`) si chiamano "Lazy" perché istanziano (disegnano) solo gli elementi visibili sullo schermo in quel momento, offrendo grandissime performance.

Listing 7.3: Creare una lista in Compose

```
1 @Composable
2 fun UserListScreen(users: List<String>) {
3     LazyColumn {
4         // Il blocco 'items' sostituisce il vecchio Adapter!
```

```

5      items(users) { user ->
6          // Qui definiamo la singola riga
7          Text(text = "Utente:␣$user", modifier = Modifier.
8              padding(8.dp))
9      }
10 }

```

7.5 Esecuzione Asincrona e Immagini (Side Effects)

7.5.1 LaunchedEffect

Se dobbiamo eseguire codice asincrono (come scaricare un JSON o dati da un database) non possiamo scriverlo direttamente nel corpo di un `@Composable`, altrimenti verrebbe rieseguito (scaricando ripetutamente i dati) ad ogni singola ricomposizione!

Per evitare questo si usa `LaunchedEffect`. Viene eseguito una sola volta quando il composable entra nello schermo (o quando una "key" specificata cambia). Per operazioni di rete si usa il blocco `withContext(Dispatchers.IO)`.

7.5.2 Caricamento Immagini da URL

In Jetpack Compose, per caricare un'immagine da Internet si consiglia la libreria di terze parti **Coil**, che offre il widget `AsyncImage`. Fa tutto da solo: apre un thread asincrono, scarica, salva in cache e mostra l'immagine, senza bloccare la UI.

Listing 7.4: Esempio di `AsyncImage`

```

1 AsyncImage(
2     model = "https://picsum.photos/id/10/200/300",
3     contentDescription = "Descrizione␣per␣accessibilit ",
4     modifier = Modifier.size(64.dp)
5 )

```

7.6 Navigazione in Compose (Navigation Compose)

Il Navigation Component   stato adattato per funzionare senza XML. Servono 3 elementi:

1. `rememberNavController()`: Per tenere traccia dello stato della navigazione.
2. `NavHost`: Il contenitore che decide quale Composable mostrare. Sostituisce il NavGraph XML specificando una `startDestination`.
3. `composable("rotta")`: Funzione che associa una stringa (la rotta) al Widget da disegnare.

Listing 7.5: Navigazione in Compose

```
1  @Composable
2  fun AppNavigation() {
3      val navController = rememberNavController()
4
5      NavHost(navController = navController, startDestination = "
6          home") {
7          composable("home") {
8              HomeScreen(onNavigate = { navController.navigate("
9                  detail") })
10         }
11         composable("detail") {
12             DetailScreen()
13         }
14     }
15 }
```

Capitolo 8

Esecuzione in Background: Services e Alternative

Un'app Android ben progettata non si limita a reagire quando l'utente guarda lo schermo, ma deve saper gestire operazioni asincrone e di lunga durata in background (come riprodurre musica, scaricare file, sincronizzare dati).

8.1 Cos'è un Service?

Un **Service** è un componente dell'applicazione progettato per eseguire operazioni di lunga durata in background. **Non fornisce un'interfaccia utente (UI).**

TRABOCCHETTO DA ESAME: Service vs Thread

Un errore comune è pensare che un Service giri in un thread separato. **Falso!** Di default, **un Service viene eseguito nel Main Thread (UI Thread)** dell'applicazione host. Se il Service deve eseguire operazioni bloccanti (es. chiamate di rete o calcoli intensivi), **devi creare tu un thread o una coroutine** all'interno del Service, altrimenti bloccherai l'interfaccia grafica scatenando un errore ANR (Application Not Responding).

8.2 Le tre tipologie di Service

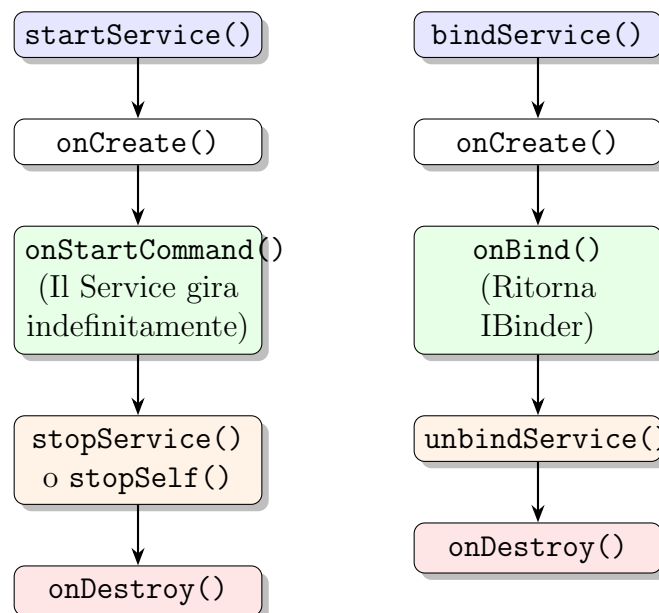
Storicamente, Android prevedeva 3 tipi di Service:

1. **Background Service:** Esegue operazioni non direttamente visibili all'utente (es. comprimere lo storage). *Attenzione:* Da Android 8.0 (API 26), il sistema impone forti limiti a questi service per risparmiare batteria, terminandoli forzatamente. Oggi si usa il **WorkManager**.

2. **Foreground Service:** Esegue operazioni **evidenti all'utente** (es. player musicale o tracciamento GPS per la corsa). Per funzionare, l'app **deve mostrare una Notifica obbligatoria** nella barra di stato, in modo che l'utente sappia che l'app sta consumando risorse.
3. **Bound Service:** Offre un'interfaccia Client-Server. Un'Activity (o un altro componente) si "lega" (bind) al Service per inviare richieste e ricevere risultati. Vive finché c'è almeno un client collegato ad esso.

8.3 Ciclo di vita: Started vs Bound

Il ciclo di vita di un Service dipende da come viene avviato dall'Activity.



8.3.1 Implementare un Started Service (es. Foreground Music Player)

Per implementare un Service che continua a suonare anche se l'app viene chiusa:

1. Si dichiara il service nel `AndroidManifest.xml` con i permessi corretti (`FOREGROUND_SERVICE` e il tipo specifico, es. `mediaPlayback`).
2. Si estende la classe `Service`.

3. Nel metodo `onStartCommand()` si lancia `startForeground(id, notification)` e si ritorna un intero che indica al sistema come comportarsi se il Service viene ucciso per mancanza di memoria:

- **START_STICKY**: Il sistema **riavvia automaticamente** il Service appena possibile, passando un intent nullo (ideale per un music player).
- **START_NOT_STICKY**: Il Service **non** viene ricreato automaticamente, a meno che non ci siano nuovi intent in coda.

Listing 8.1: Creazione di un Foreground Service

```
1 class MusicForegroundService : Service() {
2     override fun onStartCommand(intent: Intent?, flags: Int,
3         startId: Int): Int {
4         // Obbligatorio per i Foreground Service: mostrare una
5             notifica
6         startForeground(1, createNotification())
7
8         // Avvio del player musicale (es. ExoPlayer/MediaPlayer)
9         mediaPlayer?.start()
10
11         // Se il sistema uccide il service per RAM, riavvialo
12             appena possibile
13         return START_STICKY
14     }
15
16     override fun onBind(intent: Intent?): IBinder? = null // Non
17         un Bound Service
18 }
```

8.3.2 Implementare un Bound Service

I Bound Service si usano quando l'Activity deve chiamare metodi specifici del Service (es. `musicService.pause()`). Per farlo, il Service restituisce un `IBinder` tramite il metodo `onBind()`. L'Activity usa un oggetto `ServiceConnection` per ricevere l'istanza del Binder.

8.4 Modern Android: WorkManager vs Services

A causa delle severe limitazioni introdotte nelle versioni moderne di Android (Doze Mode, App Standby), i classici *Background Services* non si usano quasi più.

Oggi la regola d'oro è:

- Task **immediato** ed evidente all'utente (Musica, Navigatore)? → **Foreground Service**.
- Task **differibile** o **periodico** (es. backup giornaliero, invio log, sync dati)? → **WorkManager**.

Il **WorkManager** è una libreria Jetpack che garantisce l'esecuzione del lavoro in background anche se l'app viene chiusa o il dispositivo viene riavviato. Rispetta le ottimizzazioni della batteria e permette di impostare dei *Constraints* (es. "esegui solo se connesso in Wi-Fi e in ricarica").

8.5 Altri Componenti di Background

- **Broadcast Receiver:** Componente senza UI che reagisce a messaggi di sistema (es. Wi-Fi connesso, batteria scarica). Registrabile via Manifest o da codice (`registerReceiver`).
- **Content Provider:** Struttura per condividere dati in modo sicuro tra app diverse. Usato tipicamente per accedere al database dei Contatti o della Galleria di sistema.

Capitolo 9

Persistenza dei Dati: Room e Firebase

Un'app reale necessita di salvare dati in modo che sopravvivano alla chiusura o alla rotazione dello schermo. In Android abbiamo diverse scelte:

- **SharedPreferences / DataStore:** Per dati semplici chiave-valore (es. impostazioni, token di login).
- **File System:** Per dati non strutturati (es. foto, PDF).
- **Database (SQLite / Room):** Per dati strutturati, liste, relazioni e query complesse locali.
- **Cloud (Firebase):** Per dati condivisi tra più utenti/dispositivi.

9.1 SQLite "Puro" (Il Motore)

Android include nativamente **SQLite**, un database relazionale embedded (non c'è un processo server separato, ma una libreria C molto compatta e veloce). Caratteristica peculiare di SQLite è il *Type Affinity*: il controllo dei tipi è "rilassato" (puoi inserire una stringa in una colonna numerica).

L'approccio storico (ora considerato *obsoleto* per la troppa verbosità) richiedeva:

1. Estendere `SQLiteOpenHelper` per gestire creazione (`onCreate`) e aggiornamento schema (`onUpgrade`).
2. Scrivere query SQL manuali, prони a errori a runtime (non validati dal compilatore).

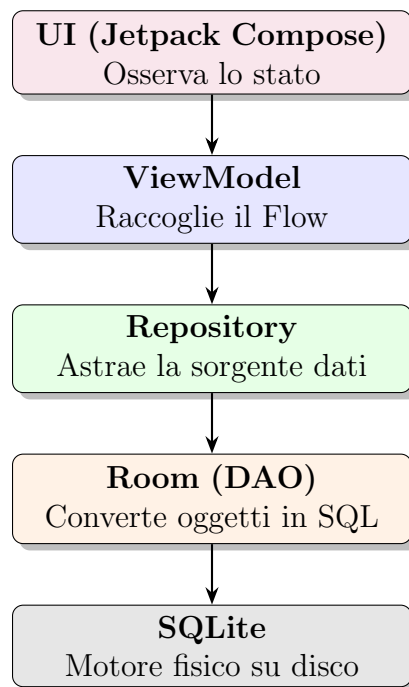
3. Usare `ContentValues` per gli INSERT e i `Cursor` (puntatori ai risultati) per leggere i dati estratti con query SELECT.

9.2 Jetpack Room: L'Approccio Moderno

Per risolvere i problemi di SQLite puro (eccesso di codice *boilerplate* e zero type-safety), Google ha introdotto **Room**, un layer di astrazione sopra SQLite.

Room è una **libreria che funziona come astrazione sopra SQLite**. Si basa su 3 elementi fondamentali:

1. **Entity:** Le data class Kotlin annotate con `@Entity`. In Room, una classe annotata con `@Entity` rappresenta **una tabella del database**.
2. **DAO (Data Access Object):** Interfacce annotate con `@Dao`. In Room, il DAO è responsabile di **gestire le query**. I metodi annotati con `suspend` indicano che la funzione **deve essere chiamata in una coroutine**. Un metodo che ritorna un `Flow` serve a **emettere dati nel tempo** (aggiornando automaticamente la UI).
3. **Database:** Una classe astratta annotata con `@Database`. Room controlla le query SQL **a compile-time**, garantendo sicurezza prima dell'esecuzione.



9.2.1 Configurazione e KSP

Per far funzionare Room, Android deve "scrivere il codice SQLite al posto tuo" durante la compilazione. Questo si fa usando il **KSP (Kotlin Symbol Processing)**. Se nel file `build.gradle.kts` dimentichi di aggiungere il plugin KSP o la dipendenza `ksp(libs.androidx.room.compiler)`, il progetto non compilerà.

9.2.2 Esempio di Codice Room

Listing 9.1: I tre componenti di Room

```
1 // 1. ENTITY (Tabella)
2 @Entity(tableName = "shopping_items")
3 data class ShoppingItem(
4     @PrimaryKey(autoGenerate = true) val id: Int = 0,
5     val name: String,
6     val quantity: Int
7 )
8
9 // 2. DAO (Query)
10 @Dao
11 interface ShoppingDao {
12     // Flow trasforma la query in uno stream Reattivo!
13     @Query("SELECT * FROM shopping_items")
14     fun getAllItems(): Flow<List<ShoppingItem>>
15
16     // suspend impone l'uso di Coroutine (no blocco UI)
17     @Insert
18     suspend fun insertItem(item: ShoppingItem)
19 }
20
21 // 3. DATABASE
22 @Database(entities = [ShoppingItem::class], version = 1)
23 abstract class AppDatabase : RoomDatabase() {
24     abstract fun shoppingDao(): ShoppingDao
25 }
```

Da Flow a UI Reattiva in Compose

Una potenza di Room + Compose è l'uso del Flow. Quando il DB viene aggiornato (es. inserimento), Room emette un nuovo Flow. Nella UI (Compose), usando la funzione `collectAsStateWithLifecycle()`, il Flow si converte in State. Al mutare dello State, Compose fa automaticamente la **Ricomposizione**, ridisegnando la lista senza bisogno di `notifyDataSetChanged()`!

9.3 Backend as a Service: Firebase

Firebase è una piattaforma di Google che fornisce servizi di backend pronti all'uso. Sostituisce la necessità di creare un proprio server, gestire database remoti e sviluppare API. Servizi principali: *Authentication*, *Cloud Storage* (per file), e i due database NoSQL.

9.3.1 Realtime Database vs Cloud Firestore

Una tematica classica da esame è la differenza tra i due database di Firebase. Entrambi offrono sincronizzazione in tempo reale tramite **WebSocket**.

Realtime Database	Cloud Firestore
I dati sono memorizzati come un grande JSON tree .	I dati sono organizzati in Collection che contengono Documenti . Un document a sua volta può contenere campi .
Meno scalabile. Scaricare un nodo scarica <i>tutti i dati</i> in esso contenuti (spesso si scaricano dati non necessari).	Più scalabile . Le query permettono il download mirato. Una query con più condizioni può richiedere la creazione di un indice .
Si usano listener come <code>ValueEventListener</code> per il tempo reale.	Il metodo <code>addSnapshotListener()</code> permette di ricevere aggiornamenti in tempo reale .

9.3.2 Denormalizzazione (Best Practice per DB NoSQL)

Nei database relazionali (SQL) si normalizzano i dati (es. Join tables). Nei database NoSQL come Firebase, la regola d'oro (per evitare di scaricare MegaBytes di messaggi solo per leggere il titolo di una chat) è **appiattare i dati (Flattening) e Denormalizzare**.

Si evitano alberi profondi creando nodi separati al livello principale (top level nodes):

Listing 9.2: Antipattern vs Denormalizzazione

```
1 // SBAGLIATO (Nidificato)
2 "classes": {
3   "class1": {
4     "students": { "student1": true } // Se chiamo 'class1'
5     scarico tutti gli studenti!
6   }
7 }
```

```

6 }
7
8 // CORRETTO (Appiattito e Denormalizzato)
9 "classes": { "class1": {...dati classe} }
10 "students": { "student1": {...dati studente} }
11 "class_enrolments": {
12     "class1": { "student1": true } // Tabella pivot usata solo per
        le query
13 }

```

9.3.3 Lettura Dati e Listener

Nei quiz è richiesto di riconoscere i listener appropriati:

- **Realtime Database:** Si usa `ValueEventListener` (che ha i metodi `onDataChange` e `onCancelled`).
- **Firestore:** Per leggere una tantum si usa `.get().addOnSuccessListener`. Per ricevere aggiornamenti *in tempo reale*, si usa `addSnapshotListener()`.

Le query in Firebase sono **Asincrone**. Non bloccano la UI. I dati ottenuti vanno poi mappati in liste. Se si usa il vecchio sistema XML con `SimpleAdapter`, bisogna ricordare che **non supporta nativamente il caricamento di immagini da URL**; per farlo occorre definire un `ViewBinder` personalizzato e usare librerie come **Glide** o **Coil**.